

CURRICULUM VITAE



# Miloš Vasić

**AI Engineer · Software Engineer**

---

LLM infrastructure, autonomous agents, and the governance that makes them trustworthy. Fleets over monoliths; anti-bluff, evidence-gated quality.

|                                  |                                         |                                       |                             |
|----------------------------------|-----------------------------------------|---------------------------------------|-----------------------------|
| <b>2009</b><br>ENGINEERING SINCE | <b>140+</b><br>REPOSITORIES<br>GOVERNED | <b>43</b><br>LLM PROVIDERS<br>UNIFIED | <b>6+</b><br>CORE LANGUAGES |
|----------------------------------|-----------------------------------------|---------------------------------------|-----------------------------|

## Ingeniero AI — LLM, agentes autónomos y la gobernanza que los hace confiables.

- Correo: milos85vasic@gmail.com
- Web: <https://milosvasic.ru> · <https://vasic.digital>
- GitHub: [vasic-digital](#) · [HelixDevelopment](#) · [Server-Factory](#)

## Resumen

Ingeniero AI/software que construye sistemas de desarrollo AI de principio a fin —desde infraestructura LLM multi-proveedor y agentes autónomos hasta las capas de control de calidad y gobernanza que garantizan su fiabilidad. No entrego demostraciones; entrego plataformas. Más de 15 años de experiencia profesional en ingeniería (desde 2009) en SDKs móviles, integración de hardware en tiempo real y backends distribuidos, que ahora convergen en un único objetivo: hacer que el desarrollo AI autónomo sea confiable a escala.

Diseño flotas, no monolitos —grandes aplicaciones de producto compuestas por docenas de módulos pequeños, desacoplados y probados de forma independiente, cada uno heredando un Constitution de ingeniería compartido y verificado mediante una disciplina de control de calidad basada en evidencia y con compuertas anti-engaño. Lenguaje principal: Go, con Kotlin/KMP, TypeScript/React, Python, Swift y Shell. Principio rector, aplicado de forma mecánica y no solo enunciado: una funcionalidad está terminada solo cuando un usuario real puede utilizarla y existe evidencia capturada que lo demuestra.

**Lo que apporto:** la capacidad de llevar una capacidad AI desde una idea de investigación hasta un sistema gobernado, autoverificable y listo para producción —enrutamiento LLM que prueba que cada modelo funciona antes de confiar en él, agentes que debaten y alcanzan consenso en lugar de adivinar, capas de memoria y RAG que no pierden contexto, y un ecosistema completo diseñado para que “las pruebas están en verde” nunca signifique en silencio “la funcionalidad está rota”.

## Competencias clave

- **Sistemas AI / LLM:** abstracción de infraestructura LLM multi-proveedor (40+ proveedores), integración de herramientas MCP, RAG, bases de datos vector y embeddings, orquestación de agentes (agentes CLI sin interfaz, flujos de trabajo en grafo, debates/consensos en múltiples rondas), planificación (HiPlan/MCTS/Árbol-de-Pensamientos), LLMOps, evaluación comparativa (SWE-bench/HumanEval/MMLU), verificación LLM, barreras de protección LLM defensivas, visión por computadora + visión LLM.
- **Ingeniería de backend:** Go (Gin), gRPC + Protobuf, HTTP/3 (QUIC), WebSockets, sistemas distribuidos (incl. especificaciones formales TLA+), servicios REST de alto rendimiento y trabajadores concurrentes.

- **Datos:** PostgreSQL, SQLite, SQLCipher (cifrado en reposo), Redis, Neo4j, ClickHouse, almacenamiento de objetos (MinIO/S3/GCS/Azure).
- **Frontend / multiplataforma:** TypeScript/React (Tailwind, Redux Toolkit, i18next), Angular, Electron, React Native, Kotlin Multiplatform, Android/Android TV (Kotlin), iOS (Swift), Tauri/Rust.
- **Infraestructura / DevOps:** Docker y Compose, Kubernetes + Helm, Prometheus + Grafana, OpenTelemetry, QEMU/Libvirt/Parallels; CI/CD mediante GitHub Actions, Gradle, Make.
- **Control de calidad / ingeniería de calidad:** control de calidad basado en evidencia y compuertas anti-engaño (HelixQA), arneses de desafío con compuertas de mutación, `go test -race`, pruebas de regresión visual, pruebas en dispositivos ADB, SonarQube, escaneo de seguridad (semgrep/gosec/trivy/snyk/gitleaks/nancy).
- **Gobernanza de ingeniería:** Constitution como submódulo; compuertas de herencia y propagación; disciplina de documentación y cobertura en una flota de más de 140 repositorios.

## Proyectos seleccionados

### Gobernanza y control de calidad

- **HelixConstitution** — un manual de ingeniería universal distribuido como submódulo de Git e heredado en más de 140 repositorios: leyes de ingeniería desplegadas y versionadas exactamente como el código. Un solo *bump* del submódulo actualiza las reglas para toda la flota, y los mecanismos de propagación revisan literalmente cada repositorio consumidor en busca de la cláusula requerida — cada comprobación emparejada con una meta-prueba de mutación que demuestra que el mecanismo en sí no es un engaño—. Convierte las “mejores prácticas que la gente espera seguir” en leyes anti-engaño heredadas, auditables y aplicadas de forma mecánica.
- **HelixQA** — orquestación de control de calidad anti-engaño (Go) basada en una regla inquebrantable: el estándar no es “las pruebas pasan”, sino “los usuarios pueden usar la función”. Ejecuta bancos de pruebas escritas YAML y sesiones de control de calidad autónomas LLM y de visión que abren la aplicación real, verifican cada función documentada, buscan errores no documentados en Android/Android TV/Web/Escritorio y se niegan a otorgar un APROBADO sin evidencia en tiempo de ejecución capturada —capturas de pantalla, *logcat*, vídeo, trazas de pila— más tickets listos para corrección AI.

### Desarrollo AI e infraestructura LLM

- **HelixAgent** — servicio LLM de nivel productivo (Go/Gin) que se niega a confiar en un solo modelo: distribuye un *prompt* entre múltiples proveedores, ejecuta debates estructurados en varias rondas (Propuesta → Crítica → Revisión → Síntesis) y enruta según puntuaciones de verificación en vivo con *fallback* gradual —todo detrás de una API compatible con OpenAI, con capa de datos de alta disponibilidad, observabilidad y salvaguardas—. *Go, Gin, PostgreSQL, Redis, Prometheus/Grafana/OpenTelemetry, MCP, Neo4j/ClickHouse/Kafka*.
- **HelixCode** — plataforma de desarrollo AI distribuida que divide el trabajo en tareas inteligentes y conscientes de dependencias en una flota de *workers* gestionada por SSH, luego realiza *checkpoints* y *rollbacks* para que nada se pierda si una tarea se interrumpe; selección de modelos basada en hardware y ciclo de vida completo de planificación/construcción/prueba/refactorización detrás de REST/CLI/TUI/MCP. *Go, Gin, PostgreSQL, Redis, SSH, MCP, llama.cpp/Ollama*.

- **HelixLLM** — un único binario, seis modos de despliegue: inferencia compatible con OpenAI y Anthropic sobre HTTP/3 que escala desde un portátil hasta un clúster multi-host, con inferencia local *llama.cpp* (CUDA/Metal/ROCm) y una cadena de *fallback* en la nube de autodescubrimiento y puntuación por verificación que siempre degrada a un modelo local garantizado. *Go*, *HTTP/3 QUIC*, *gRPC/SSE/Kafka*, *llama.cpp*.
- **LLMProvider / LLMOrchestrator / LLMsVerifier** — la columna vertebral de la infraestructura LLM: una única interfaz sobre 43 proveedores con *circuit breakers*, monitoreo de salud, reintentos con *backoff* aleatorio y descubrimiento de modelos honesto (sin *fallbacks* codificados); un plano de control *thread-safe* que genera y dirige agentes CLI sin interfaz gráfica (OpenCode, Claude Code, Gemini, Junie, Qwen Code) mediante un protocolo híbrido de tuberías y archivos; y una fuente de verdad de verificación cuyo *gate* obligatorio “¿Ves mi código?” garantiza que solo los modelos probados como funcionales se marquen como utilizables o se exporten.
- **HelixMemory / HelixSpecifier** — un motor unificado de memoria cognitiva que fusiona cuatro *backends* de primera categoría (Mem0, Cogneer, Letta, Graphiti) bajo una única interfaz con búsqueda paralela y reordenamiento entre fuentes; y un motor de fusión para desarrollo basado en especificaciones que ajusta su propio protocolo a la escala del trabajo y respalda las especificaciones con debates multiagente.
- **HelixTrack** — una alternativa JIRA + Confluence de código abierto (buque insignia de la línea Helix-Track): microservicios Go con una API unificada y enrutada por acciones sobre HTTP/3, cifrado SQLCipher en reposo y clientes nativos para Web/Escritorio/Android/iOS.

## Herramientas de nivel profesional (utilidades vasic-digitales)

- **Catalogizer** — gestión de colecciones multimedia con soporte multiprotocolo, cifrado y autoalojamiento (Go/Gin + React), resistente a almacenamiento en red inestable, construida sobre 21 submódulos reutilizables.
- **Courses-Creator** — tubería de cursos AI de Markdown a vídeo con TTS y reproductores para escritorio, móvil y web.
- **VisionEngine** — percepción de interfaz con visión por computadora y LLM-vision de múltiples proveedores, junto con grafos de navegación.
- **DocProcessor · Docs Chain · Herald · task\_bridge · Vasic Digital Suite de Módulos Reutilizables** — mapeo de características para control de calidad, sincronización de documentos/bases de datos con hash de contenido, notificaciones en lenguaje natural, sincronización de tareas/tableros y la flota de la biblioteca estándar `digital.vasic.*`.

## Automatización de infraestructura (Server Factory)

- **Mail Server Factory** — JSON declarativo → servidores de correo completamente aprovisionados y dockerizados para 12 tipos de conexión y 25 distribuciones Linux; reporta 439 pruebas superadas y una puerta SonarQube limpia.
- **Server Factory Marco Central, Qemu-Utils, Parallels-Utils** — el motor de aprovisionamiento compartido y las herramientas para imágenes de máquinas virtuales.

## Lenguajes y herramientas (lista rápida)

Go · Kotlin · Kotlin Multiplatform · TypeScript · JavaScript · Python · Swift · Java · Rust · Shell · PL/pgSQL · TLA+ · Gin · gRPC · HTTP/3 · React · Angular · Electron · React Native · PostgreSQL · SQLite · SQLCipher · Redis · Neo4j · ClickHouse · Docker · Kubernetes · Prometheus · Grafana · OpenTelemetry · QEMU · GitHub Actions · Gradle · Make

## Experiencia

*Ingeniero de software desde 2009, con experiencia en todo el ciclo de desarrollo: planificación, desarrollo, liderazgo de equipos e implementación. El historial completo a continuación proviene del registro verificado del candidato (milosvasic.ru).*

### Puestos a tiempo completo

- **Desarrollador SDK — Harness** (harness.io), Belgrado, Serbia · 03/2020 – 12/2024. Desarrollador principal en la familia de SDKs para la división de Feature Flag de la empresa, enfocado en todas las principales plataformas móviles y más allá. Clientes y socios incluyeron AWS, Google y varios bancos. *Tecnologías: Android, iOS, Flutter, React Native, TypeScript, JavaScript, Java, Kotlin, Swift, Go, Ruby.*
- **Ingeniero de Software — Leica Geosystems** (leica-geosystems.com), Heerbrugg, Suiza · 02/2016 – 02/2020. Ingeniería principalmente en iOS y Android para los escáneres 3D de vanguardia de Leica Geosystems: comunicación en tiempo real con el hardware, procesamiento de datos y sincronización. Socio: Autodesk. *Tecnologías: Android, iOS, Java, Kotlin, Swift, C++.*
- **Desarrollador SDK — Bosch** (bosch.rs), Belgrado, Serbia · 01/2010 – 01/2016. Desarrollador principal de SDK para el proyecto Vehículos Conectados: comunicación Bluetooth en tiempo real con el bus OBD2, procesamiento de datos de alto rendimiento y persistencia. *Tecnologías: Android, Java, Kotlin.*

### Otros compromisos

- **TN-TECH** (tn-tech.co.rs), Novi Sad, Serbia · media jornada, desde 03/2017. Trabajo para Globex Data (Canadá y Suiza) — Sekur (SekurMessenger), SekurMail, SekurSuite — y la plataforma BusRide. *Tecnologías: Android, Java, Kotlin, C++, Qt.*
- **Increment Loop** (incrementloop.com), Belgrado, Serbia · media jornada, desde 09/2023. La aplicación Yuno. *Tecnologías: Android, Kotlin.*
- **Organizaciones propias / de código abierto** — HelixTrack, Server Factory (Mail Server Factory, Parallels-Utils, Qemu-Utils) y Vasic Digital (Android-Toolkit, Network-Binder), detalladas en Proyectos seleccionados más arriba.

## Publicaciones

- **Fundamentos de Kotlin** — autor autopublicado; última edición revisada en septiembre de 2022 (*Fundamentos de Kotlin*, 3.<sup>a</sup> edición). También autor para Packt Publishing (Reino Unido).

## Formación académica

- **Máster en Tecnologías de la Información Contemporáneas** — Universidad Singidunum, Belgrado, Serbia · 2014.
- **Grado en Informática y Computación** — Universidad Singidunum, Belgrado, Serbia · 2008.